# DP-OPS-0056: Architectural Hardening of the Stela Task Subsystem and Governance Constitution

## 1. Executive Summary and Strategic Context

The operational stability of the Stela repository depends on a rigorous, deterministic interaction between the human Operator and the AI workforce. This relationship is mediated by the TASK.md surface—a living dashboard that serves simultaneously as a status report, a command interface, and a historical log. Recent operational cycles have exposed structural fragilities in this subsystem, specifically regarding the synchronization between the active work surface and its enforcing binaries.[1] The reliance on brittle, text-dependent validation logic has introduced friction where minor formatting deviations trigger disproportionate system halts, effectively blocking valid work due to "ghost canon" drift.[1]

This report articulates a comprehensive architectural proposal for **DP-OPS-0056**, aiming to solidify the Task Subsystem by decoupling schema validation from dispatch mechanics, enforcing rigorous idempotency in hygiene routines, and establishing a "perfect and pristine" contract surface. A central tenet of this refactoring is the immediate and total excision of legacy versioning terminology (specifically "V2" nomenclature) to reinforce the "Pointer-First" constitution: the repository contains only one active truth, and it does not version-control its own laws within living filenames.

The analysis synthesizes best practices in shell scripting—specifically modularity, fail-fast error handling, and idempotent state management—to propose a rewritten suite of maintenance tools (task.sh, dp.sh, prune).[2] Furthermore, it delivers the required technical specifications for docs/ops/specs/tools/lint/dp.md and docs/ops/specs/tools/lint/task.md, ensuring that the technical documentation matches the executable reality. The outcome is a governance system where TASK.md remains mathematically predictable, linters are mutually exclusive yet collectively exhaustive, and the pruning mechanism operates with absolute safety.

---

## 2. Governance Philosophy: The Pointer-First Constitution

The "Pointer-First" constitution is not merely a stylistic choice; it is a mechanism for enforcing epistemological integrity in a stateless agent environment. By prohibiting duplicated canon text, the system forces the agent to retrieve truth from the source rather than relying on

potentially stale summaries.

### 2.1 The Danger of Duplicated Canon

In previous iterations, TASK.md contained summarized instructions derived from PoT.md. This redundancy created a "Split-Brain" hazard. If PoT.md was updated but the summary in TASK.md was not, an agent reading the Task Dashboard would operate on obsolete logic.[1]

- **The Fix:** TASK.md must contain *only* pointers. Instead of listing "Do not use contractions," it must link to PoT.md Section 4.2. This minimizes token usage in the context window—a critical optimization for agent performance [4]—and guarantees that the agent is referencing the absolute Single Source of Truth (SSOT).

### 2.2 Removing "Ghost Canon" (The "V2" Prohibition)

The presence of "V2", "v2", or "TASK V2" in filenames or script logic implies the existence of a "V1" (deprecated) or a "V3" (future). In a continuous delivery environment governed by a Policy of Truth (PoT.md), there is only *the* current state.

- **Implication:** Version numbers on living governance files introduce ambiguity. An agent might hallucinate a difference between "TASK V2 rules" and "Current Rules."
- **Mandate:** All instances of versioning nomenclature must be stripped. The file is simply TASK.md. The spec is task.md. The logic is "Current." This flattens the ontology and removes the cognitive load of version resolution from the agent.[1]

### 2.3 The Staffing Protocol and Authority

The staffing protocol defined in the user query establishes a clear hierarchy: Operator (Decision), Integrator (Governance), Contractor (Execution).

- **Algorithmic Enforcement:** The linters proposed herein enforce this hierarchy. tools/lint/task.sh acts as the proxy for the Integrator, rejecting any Contractor output that violates the governance structure. ops/bin/prune acts as the Operator's hygiene agent, strictly enforcing clean state without making creative decisions.

---

## 3. System Pathology and Failure Analysis

To engineer a robust solution, one must diagnose the specific pathologies within the current TASK.md ecosystem. Analysis of the repository dumps [1] reveals three primary failure modes.

### 3.1 The "Split-Brain" Linter Problem

Currently, both tools/lint/task.sh and tools/lint/dp.sh perform overlapping validation on the TASK.md header structure.

- **Redundancy:** Both scripts check for the presence of specific headers, but often use

slightly different regex patterns.

- **Brittleness:** A slight deviation (e.g., ## 1. Session State vs ## 1. Session State) might pass one linter but fail the other. This violation of the DRY (Don't Repeat Yourself) principle leads to operator confusion and "lint fatigue".[1]
- **Ambiguity:** It is unclear whether dp.sh validates the *content* of the task (the specific branch name) or the *format* (the existence of a branch field).

### 3.2 The Mutation Hazard in prune

The ops/bin/prune binary currently conflates "scrubbing" (resetting to a template) with "pruning" (removing garbage).

- **Risk:** If prune attempts to "fix" a broken TASK.md by rewriting sections based on assumptions, it creates a mutation risk. If the assumptions are wrong (e.g., due to a template mismatch), prune effectively destroys active work or corrupts the governance history.[1]
- **Requirement:** Hygiene tools must be idempotent and non-mutating regarding semantic content. They should remove waste, not rewrite law.

### 3.3 The "Lazy Context" Hazard

The notes for DP-OPS-0056 highlight a tendency for Dispatch Packets to reference external "chat logs" or "scratch files" as required context.[1]

- **The Hazard:** This breaks the "Self-Contained Repository" invariant. If an agent requires a file outside the repo to understand its instructions, the work is not reproducible.
- **Resolution:** The dp.sh linter must aggressively reject any DP that references storage/_scratch or untracked artifact paths in its Required Context or Scope sections.

---

## 4. Architectural Design Standards: The "Perfect" Subsystem

The proposed subsystem utilizes strict Separation of Concerns (SoC) and robust coding patterns to resolve the pathologies identified above.

### 4.1 Separation of Concerns (SoC) Matrix

| Component | Role | Jurisdiction | Enforcement Logic |
|---|---|---|---|
| **TASK.md** | **The Container** | Active Dashboard | Structural Schema |

| | | | |
|---|---|---|---|
| **tools/lint/task.sh** | **Schema Authority** | Validates the *container* structure. | Ensures TASK.md has correct headers, pointers, and log sections. Ignores DP content. |
| **tools/lint/dp.sh** | **Transaction Authority** | Validates the *payload* (DP). | Ensures the active DP has valid branches, freshness gates, and receipts. |
| **ops/bin/prune** | **Hygiene Engine** | Artifact Lifecycle | Removes waste. Resets container to template *only* when explicitly requested. Never "fixes" in place. |

### 4.2 Algorithmic Rigor and Best Practices

The new tools will adhere to high-integrity shell scripting standards [2]:

- **Fail Fast:** All scripts must utilize set -euo pipefail to ensure immediate exit on error, undefined variables, or pipe failures.
- **Guard Clauses:** Functions must verify preconditions (e.g., file existence) before attempting operations.
- **Regex Rigor:** Utilization of Extended Regular Expressions (grep -E) to handle variable whitespace and case insensitivity where semantic meaning is unaffected. This moves validation from *Syntactic Strictness* (fragile) to *Semantic Correctness* (robust).
- **Idempotency:** Operations like pruning must be repeatable without side effects. If a file is already gone, prune should report success, not failure.[7]

---

## 5. Technical Specification: TASK.md (The Living Surface)

This section defines the "Perfect/Pristine" structure for the TASK.md file. This definition is the baseline for the docs/ops/specs/surfaces/task.md deliverable.

### 5.1 The Immutable Skeleton

The file is divided into five immutable sections. These headers must exist exactly as specified (tolerant of spacing) for the file to be valid.

1. **## 1. Session State**: A pointer-only block.
    - *Constraint:* Must link to the OPEN artifact generated by ops/bin/open. No inline branch names.
2. **## 2. Logic Pointers**: The legal framework.
    - *Constraint:* Must explicitly link PoT.md and docs/MANUAL.md.
3. **## 3. Current Dispatch Packet**: The active transaction slot.
    - *Constraint:* Must contain the Freshness Gate, Scope, and Execution Plan subsections.
4. **## 4. Closeout**: The mandatory shutdown protocol.
    - *Constraint:* Must verify the Closeout Cycle defined in docs/MANUAL.md.
5. **## 5. Work Log**: The append-only history.
    - *Constraint:* Must end with a sentinel value (No active thread) when idle.

### 5.2 The "V2" Removal

All references to "V2" in the header or status block are removed. The file is versioned by git, not by its own content. The header is simply # STELA TASK DASHBOARD.

### 5.3 Data Flow

1. **Operator** runs ops/bin/open. This generates storage/handoff/OPEN-main-12345.txt.
2. **Operator** updates TASK.md Section 1 to point to this new artifact.
3. **Operator** drafts a DP in Section 3.
4. **Operator** runs tools/lint/dp.sh to validate the draft.
5. **Agent** reads TASK.md, executes work, and updates Section 5 with a log entry.
6. **Agent** runs ops/bin/prune --scrub to reset Section 3 to blank, ready for the next cycle.

---

## 6. Technical Specification: tools/lint/task.sh

This script is the **Schema Enforcer**. It ensures TASK.md is a valid *container*.

### 6.1 Logic Flow

1. **Preamble:** set -euo pipefail. define die() function for error reporting.[3]
2. **Existence Check:** Verify TASK.md exists at repo root.
3. **Forbidden Term Scan:** grep -i "TASK V2\|V2\|v2" to ensure no versioning artifacts remain. Fail immediately if found.
4. **Block Validation (State Machine):**
    - Initialize expected_sections array.
    - Read file line-by-line. Match headers against expected sequence.
    - *Check:* ## 1. Session State must appear before ## 2. Logic Pointers.

    - *Check:* ## 1. Session State must contain Pointer: storage/handoff/OPEN-.*.
    - *Check:* ## 2. Logic Pointers must contain PoT.md.
5. **Sentinel Check:** Verify the last lines of Section 5 contain either a timestamped entry or the idle sentinel (No active thread).
6. **Output:** "OK: TASK Schema Valid" or specific error message identifying the missing/malformed section.

### 6.2 Code Standards

- Use awk for block extraction to avoid multi-pass reads.
- Use local variables to avoid namespace pollution.
- Exit code 0 for success, 1 for schema violation, 2 for missing file.

---

## 7. Technical Specification: tools/lint/dp.sh

This script is the **Transaction Enforcer**. It ensures the work inside Section 3 is valid.

### 7.1 Logic Flow

1. **Extraction:** Extract Section 3 from TASK.md (or target draft file) into a temporary buffer.
2. **Freshness Gate Validation:**
    - Parse Base HEAD. Assert it matches [0-9a-f]{7,} (Git short hash).
    - Parse Required Work Branch. Assert it matches work/dp-ops-[0-9]{4}-.*.
    - *Anti-Pattern:* If Base HEAD is "latest" or "TBD", fail. It must be specific.[9]
3. **Source of Truth Hardening:**
    - Scan for forbidden keywords: chat log, conversation dump, storage/_scratch.
    - Fail if these are found in the Required Context or Scope sections. Agents must use ops/bin/dump artifacts, not chat history.[1]
4. **Execution Plan Verification:**
    - Verify subsection 3.4.5 Receipt.
    - Check for mandatory verification commands: bash tools/lint/task.sh, bash tools/lint/dp.sh.
    - Check for the Mandatory Closing Block template.
5. **Output:** "OK: DP Valid" or detailed list of missing requirements.

---

## 8. Technical Specification: ops/bin/prune

This script is the **Hygiene Engine**. It removes waste and resets state.

### 8.1 Logic Flow

1. **Mode Selection:** Detect --scrub or --dp=ID flags.

2. **Scrub (Reset) Algorithm:**
    - **Load Template:** Read ops/lib/templates/TASK_TEMPLATE.md (or internal heredoc).
    - **Preserve History:** Extract Section 5 (Work Log) from the current TASK.md.
    - **Combine:** Concatenate Template Header + Template Sections 1-4 + Preserved Section 5.
    - **Atomic Write:** Write to storage/tmp/TASK.new. Run tools/lint/task.sh storage/tmp/TASK.new.
    - **Commit:** If lint passes, mv storage/tmp/TASK.new TASK.md. This guarantees prune never breaks the dashboard.[6]
3. **Prune (Cleanup) Algorithm:**
    - Require --dp argument to target specific artifacts.
    - Use find to locate files in storage/handoff and storage/dumps matching the DP ID.
    - Delete matches using rm -f (idempotent).[8]
    - Report count of deleted files.

---

## 9. Deliverable: docs/ops/specs/tools/lint/task.md

# Technical Specification: tools/lint/task.sh

## 1. Overview

The task.sh linter is the **Schema Authority** for the Stela repository. It enforces the structural integrity of the TASK.md dashboard, ensuring it remains a valid container for agent operations. It adheres strictly to the "Pointer-First" constitution.

## 2. Jurisdiction

- **Target:** TASK.md (Repository Root).
- **Authority:** PoT.md Section 4 (Staffing & Logic).
- **Scope:** Validates headers, section ordering, pointer existence, and log sentinels.
- **Exclusion:** Does not validate the specific content of the Dispatch Packet (see dp.sh).

## 3. Validation Standards

### 3.1 Structural Schema

The file must contain the following high-level sections in exact order:

1. ## 1. Session State
2. ## 2. Logic Pointers
3. ## 3. Current Dispatch Packet
4. ## 4. Closeout

5. ## 5. Work Log

**Regex Rigor:** Validation uses case-insensitive extended regex to tolerate minor spacing variations (^##[[:space:]]+1\.[[:space:]]+Session State).

### 3.2 Pointer Hygiene

To prevent "Ghost Canon," specific pointers are mandatory:

- **Session State:** Must point to ops/lib/manifests/CONTEXT.md.
- **Logic Pointers:** Must link PoT.md, docs/MANUAL.md, and docs/ops/specs/surfaces/task.md.
- **Prohibition:** The terms "V2", "v2", or "TASK V2" are strictly forbidden.

### 3.3 Log Integrity

The Work Log section must end with the sentinel (No active thread) or a valid timestamped entry. This ensures the agent knows the state of the previous thread.

## 4. Error Codes

- 0: Success.
- 1: Schema Violation (Missing/Disordered Headers).
- 2: Pointer Violation (Missing Links).
- 3: Drift Violation (Forbidden Terms).

---

## 10. Deliverable: docs/ops/specs/tools/lint/dp.md

# Technical Specification: tools/lint/dp.md

## 1. Overview

The dp.sh linter is the **Transaction Authority**. It validates the mechanics of the Dispatch Packet (DP) to ensure instructions are executable, safe, and reproducible.

## 2. Jurisdiction

- **Target:** TASK.md (Section 3) or draft DP files.
- **Authority:** TASK.md Contract and PoT.md Dispatch Protocols.

## 3. Validation Standards

### 3.1 Freshness Gate

The DP must define a clear starting state:

- **Base Branch:** Must be defined.
- **Base HEAD:** Must be a valid git short-hash (7+ hex characters). "Latest" is rejected.
- **Work Branch:** Must follow the schema work/dp-ops-XXXX-YYYY-MM-DD.

### 3.2 Source of Truth Hardening

To prevent "lazy context" hallucinations:

- **Prohibition:** References to storage/_scratch, external chat logs, or pasted dumps are forbidden in Required Context or Scope.
- **Requirement:** Context must be loaded via ops/bin/open or explicit repo paths.

### 3.3 Execution Plan

The plan must be verifiable:

- **Receipts:** Section 3.4.5 must list mandatory verification commands:
  - bash tools/lint/task.sh
  - bash tools/lint/dp.sh
- **Closeout:** Must contain the Mandatory Closing Block scaffolding.

## 4. Error Codes

- 0: Valid DP.
- 1: Critical Defect (Missing Gates, Unsafe References).
- 2: Schema Mismatch (Malformed Structure).

---

# 11. Deliverable: docs/ops/specs/binaries/prune.md

# Technical Specification: ops/bin/prune

## 1. Overview

ops/bin/prune is the Hygiene Engine. It manages artifact lifecycles and resets the TASK.md dashboard. It operates on a strict "Hygiene only, no Mutation" policy.

## 2. Modes

### 2.1 Prune (Cleanup)

- **Flag:** --dp=DP-ID
- **Action:** Deletes files in storage/handoff and storage/dumps containing the ID.

- **Safety:** Uses idempotent rm -f. Does not fail if files are missing.

### 2.2 Scrub (Reset)

- **Flag:** --scrub
- **Action:** Resets TASK.md to the canonical template.
- **Algorithm:**
  1. Read current Work Log from TASK.md.
  2. Load static template.
  3. Combine template + preserved log.
  4. Verify new content with tools/lint/task.sh.
  5. Overwrite TASK.md *only* if verification passes.

## 3. Usage

- ./ops/bin/prune --dp=DP-OPS-XXXX (Cleanup artifacts)
- ./ops/bin/prune --scrub (Reset dashboard)

---

## 12. Deliverable: docs/ops/specs/surfaces/task.md

# TASK Surface Contract

## Purpose

TASK.md is the canonical task-routing and DP contract surface. It is a "Pointer-First" dashboard that minimizes inline context in favor of immutable links to PoT.md.

## Surface Roles

- **Container:** The file structure enforced by tools/lint/task.sh.
- **Payload:** The active Dispatch Packet (Section 3) enforced by tools/lint/dp.sh.
- **Log:** The append-only history (Section 5) preserved by ops/bin/prune.

## Invariants

1. **No Ghost Canon:** No version numbers ("V2") allowed.
2. **Pointer-First:** Session State must be a file pointer, not text.
3. **Equilibrium:** A task is complete only when receipts are verified and the log is updated.

---

## 13. Proposal for TASK.md Refinement (Content)

The following structure represents the "Perfect/Pristine" state for TASK.md. This content will be used by ops/bin/prune as the scrub template.

# STELA TASK DASHBOARD

Status: ACTIVE

Owner: Integrator

Last Updated: (Dynamic Date)

## 1. Session State

Pointer: (none)

Context Manifest: ops/lib/manifests/CONTEXT.md

## 2. Logic Pointers

Primary Constraint: PoT.md

Surface Contract Spec: docs/ops/specs/surfaces/task.md

- Governance: docs/MANUAL.md
- Operations: ops/bin/README.md

## 3. Current Dispatch Packet (DP)

### DP-XXXX: (Template)

### 3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/dp-ops-XXXX-YYYY-MM-DD

Base HEAD: 0000000

### 3.2 Required Context Load

1. PoT.md
2. SoP.md
3. TASK.md
4. docs/MAP.md
5. docs/MANUAL.md
6. ops/lib/manifests/CONTEXT.md

### 3.3 Scope and Safety

Objective: Populate during execution.

In scope: Populate during execution.

Out of scope: Populate during execution.

Target Files allowlist: Populate during execution.

### 3.4 Execution Plan

#### 3.4.1 State

Populate during execution.

#### 3.4.2 Request

Populate during execution.

#### 3.4.3 Changelog

Populate during execution.

#### 3.4.4 Patch / Diff

Populate during execution.

#### 3.4.5 Receipt (Proofs to collect)

- Artifacts captured: OPEN, DUMP, RESULTS.
- Gates run: bash tools/lint/task.sh, bash tools/lint/dp.sh TASK.md.

## 4. Closeout

- Execute docs/MANUAL.md Closeout Cycle.
- Mandatory Closing Block (Commit, PR Title, Description).

## 5. Work Log

(No active thread)

---

## 14. Conclusion and Risk Assessment

This proposal fundamentally hardens the Stela governance model. By removing "V2" terminology, we eliminate the semantic ambiguity of versioned truth. By separating task.sh

(Structure) from dp.sh (Transaction), we eliminate the "Split-Brain" validation failures. By rewriting prune to be idempotent and atomic, we eliminate the risk of accidental repository mutation.

**Risk:** The strictness of the new regex validation may initially increase the failure rate of drafted DPs.

**Mitigation:** The explicit instruction to run bash tools/lint/dp.sh --test in the Preflight Gate provides an immediate feedback loop for agents, allowing them to self-correct formatting errors before engaging in substantive work.

This architecture satisfies the requirement for a "perfect/pristine" dashboard and sets the standard for technical excellence in agentic operations.

### Works cited


1. DP-OPS-0056-NOTES.md
2. Shell Scripting Best Practices | Cycle.io, accessed February 12, 2026, https://cycle.io/learn/shell-scripting-best-practices
3. Title: Error Handling in Shell Scripting: A Guide for DevOps Engineers - Medium, accessed February 12, 2026, https://medium.com/@syedaumerubab16/title-error-handling-in-shell-scripting-a-guide-for-devops-engineers-b1f0496ed16e
4. How we solved the "AI agent black box" problem with typed tasks | Augment Code, accessed February 12, 2026, https://www.augmentcode.com/blog/how-we-built-tasklist
5. Shell scripting best practices : r/devops - Reddit, accessed February 12, 2026, https://www.reddit.com/r/devops/comments/7baj4c/shell_scripting_best_practices/
6. Mastering Error Handling in Shell Scripts: A Guide for DevOps Engineers - DEV Community, accessed February 12, 2026, https://dev.to/574n13y/mastering-error-handling-in-shell-scripts-a-guide-for-devops-engineers-3hdn
7. How to write idempotent Bash scripts · Fatih Arslan - Reddit, accessed February 12, 2026, https://www.reddit.com/r/bash/comments/rc9y3k/how_to_write_idempotent_bash_scripts_fatih_arslan/
8. How to write idempotent Bash scripts - Fatih Arslan, accessed February 12, 2026, https://arslan.io/2019/07/03/how-to-write-idempotent-bash-scripts/
9. How Do You Write Idempotent Bash Deployment Scripts (With ShellCheck)?, accessed February 12, 2026, https://dev.co/idempotent-bash-deployment-scripts-shellcheck